

MODULE 2 — READING AND WRITING FILES IN JAVA (`java.io` PACKAGE)

Learning Objectives

At the end of this module, students should be able to:

1. Understand how to write and read data from files in Java.
2. Use `FileWriter` and `FileReader` for text-based input/output.
3. Use buffered streams (`BufferedReader` and `BufferedWriter`) for efficient file handling.
4. Append data to existing files.
5. Handle exceptions properly during file input/output operations.

2.1 Understanding Java File Streams

Definition:

A **stream** in Java represents a flow of data. File I/O uses **streams** to read data from or write data to files.

- **Input Stream** → Reads data from a file (into the program).
- **Output Stream** → Writes data to a file (from the program).

◆ Types of Streams

Type	Example Classes	Description
Character Streams	<code>FileReader</code> , <code>FileWriter</code>	Handle text data (characters).
Byte Streams	<code>FileInputStream</code> , <code>FileOutputStream</code>	Handle binary data (images, audio, etc.).

In this module, we'll focus on **Character Streams** for text files.



2.2 Writing to a File Using `FileWriter`

The `FileWriter` class is used to **write text data** to files.



Steps to Write a File

1. Create a `FileWriter` object.
2. Use the `write()` method to write data.
3. Close the stream with `close()` .



Example 1: Writing Simple Text to a File

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteFileExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("message.txt");
            writer.write("Hello Java World!\n");
            writer.write("File handling is powerful and simple.");
            writer.close();
            System.out.println("Successfully wrote to the file.");
        } catch (IOException e) {
            System.out.println("An error occurred while writing to the file.");
            e.printStackTrace();
        }
    }
}
```

Explanation:

- The `FileWriter` object creates or opens `message.txt` .
- `write()` adds data into it.
- `close()` finalizes and saves the content.

Appending to a File

To prevent overwriting existing data, pass `true` as the second argument to the `FileWriter` constructor.

```
FileWriter writer = new FileWriter("message.txt", true);
writer.write("\nThis line is appended!");
writer.close();
```

✔ **Tip:** Always close file streams to avoid data loss.

2.3 Reading a File Using `FileReader`

The `FileReader` class reads characters from a text file.

Example 2: Reading a File Character by Character

```
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("message.txt");
            int data;
            while ((data = reader.read()) != -1) {
                System.out.print((char) data);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("An error occurred while reading the file.");
            e.printStackTrace();
        }
    }
}
```

Explanation:

- The `read()` method returns each character as an integer (ASCII code).
- `-1` means the end of the file (EOF).
- Casting `data` to `(char)` converts it back to text.

2.4 Reading and Writing Efficiently with Buffered Streams

The `BufferedReader` and `BufferedWriter` classes make file operations **faster and more efficient** by using internal buffers.

BufferedWriter

Used for writing text quickly and efficiently.

Example 3: Using `BufferedWriter`

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BufferedWriteExample {
    public static void main(String[] args) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("data.txt", true))) {
            writer.write("BufferedWriter makes writing faster!\n");
            writer.write("It uses an internal buffer before saving to disk.");
            System.out.println("Data successfully written using BufferedWriter.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- The `BufferedWriter` temporarily stores written data in memory, then writes it in chunks to the file (reducing disk I/O).
- The `try-with-resources` statement automatically closes the stream.

BufferedReader

Used for reading text efficiently, line by line.

Example 4: Using BufferedReader

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class BufferedReadExample {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `readLine()` reads one line at a time.
- When the file ends, it returns `null`.
- This method is faster and more memory-efficient for larger files.

2.5 Using the Scanner Class for Reading Files

The `Scanner` class (from `java.util`) can also be used to read files — especially useful when parsing structured text or reading line-by-line easily.

Example 5: Reading File Using Scanner

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ScannerFileRead {
    public static void main(String[] args) {
        try {
            File file = new File("data.txt");
            Scanner reader = new Scanner(file);
            while (reader.hasNextLine()) {
                String line = reader.nextLine();
                System.out.println(line);
            }
            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
        }
    }
}
```

Key Points:

- Scanner provides simple methods like `nextLine()`, `nextInt()`, `nextDouble()`, etc.
- It automatically breaks text into tokens (words, numbers, etc.) if needed.



2.6 Exception Handling in File I/O

Most file I/O operations can throw `IOException` or `FileNotFoundException`.

Always handle these using `try-catch` blocks.

Example:

```
try {
    FileReader reader = new FileReader("missing.txt");
} catch (FileNotFoundException e) {
    System.out.println("Error: File does not exist.");
}
```



2.7 Try-with-Resources Statement

Java's `try-with-resources` automatically closes the file after use — no need for manual `close()` .

Example:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class TryWithResources {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
            String line;
            while ((line = reader.readLine()) != null)
                System.out.println(line);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

✓ Always prefer try-with-resources for cleaner and safer code.



2.8 FileWriter vs BufferedWriter

Feature	FileWriter	BufferedWriter
Speed	Slower	Faster
Buffering	No	Yes
Suitable for	Small data	Large or frequent writes
Example	<code>writer.write("Hello");</code>	<code>bufferedWriter.write("Hello");</code>



2.9 Classwork / Practical Exercises

1. **Write Program:** Create a file named `students.txt` and write 5 student names into it.
2. **Read Program:** Read the content of `students.txt` and print each name on a new line.

3. **Append Program:** Append 2 new names to the same file.
4. **Buffered I/O:** Rewrite Exercise 1 and 2 using `BufferedWriter` and `BufferedReader`.
5. **Word Count:** Read `students.txt` and count how many words it contains.

2.10 Challenge (Optional Project)

Create a program that allows the user to input a **note**, saves it into a file called `notes.txt`, and then displays the contents of all saved notes.

Hint:

- Use `Scanner` for input.
- Use `FileWriter(true)` to append notes.

2.11 Summary of Module 2

Concept	Class/Method	Description
Writing	<code>FileWriter.write()</code>	Writes text data
Appending	<code>FileWriter(true)</code>	Adds to existing file
Reading (basic)	<code>FileReader.read()</code>	Reads characters
Buffered Writing	<code>BufferedWriter.write()</code>	Fast writing with buffer
Buffered Reading	<code>BufferedReader.readLine()</code>	Reads file line-by-line
Reading (scanner)	<code>Scanner.nextLine()</code>	Simple reading using tokens
Exception Handling	<code>try-catch</code> , <code>try-with-resources</code>	Prevents crashes from I/O errors

End of Module 2 Summary

In this module, you learned:

- How to read and write data in files using Java I/O streams.
- The difference between normal and buffered I/O.
- How to append, read line-by-line, and handle exceptions properly.

In **Module 3**, we'll explore the `java.nio.file` **package** — a modern and more powerful way of working with files (copying, moving, deleting, and getting metadata).